

Compiled by ab

Computer Graphics (CSIT 401)

IV semester, BScCSIT

Compiled by **Ankit Bhattarai**
Ambition Guru College

Syllabus

Unit	Contents	Hours	Remarks
1.	Introduction to Computer Graphics	5	
2.	Scan Conversion Algorithms	7	<i>Very Important</i>
3.	2D Transformations and Viewing	6	<i>Very Important</i>
4.	3D Transformations and Viewing	5	
5.	3D Object Representation	6	
6.	Solid Modeling	3	
7.	Visible Surface Detection	5	
8.	Illumination, Shading and Rendering Techniques	5	
9.	OpenGL and Graphics Programming	3	

Unit 7

Visible Surface Detection

(5 hrs.)

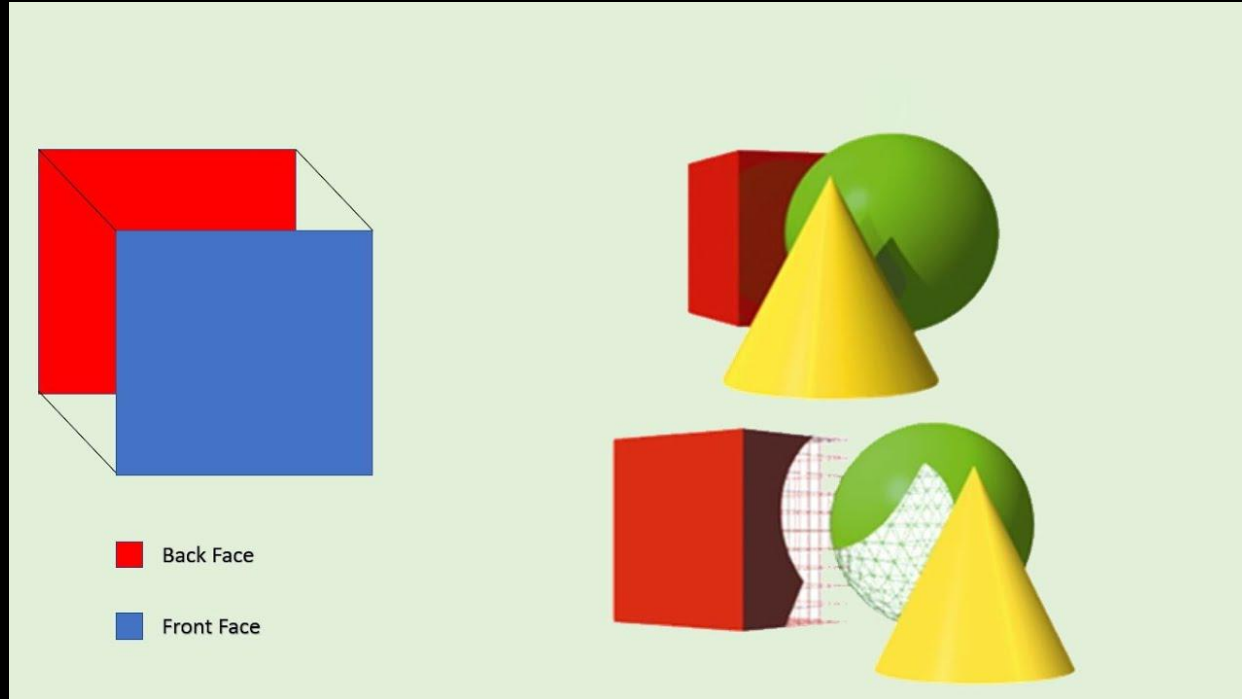
Hidden Surface Problems, Image Space and Object Space Techniques, Back Face Detection, Depth Buffer (Z - buffer), A- Buffer, Scan-Line Algorithms, Depth Sorting Method (Painter's Algorithm)

Compiled by ab

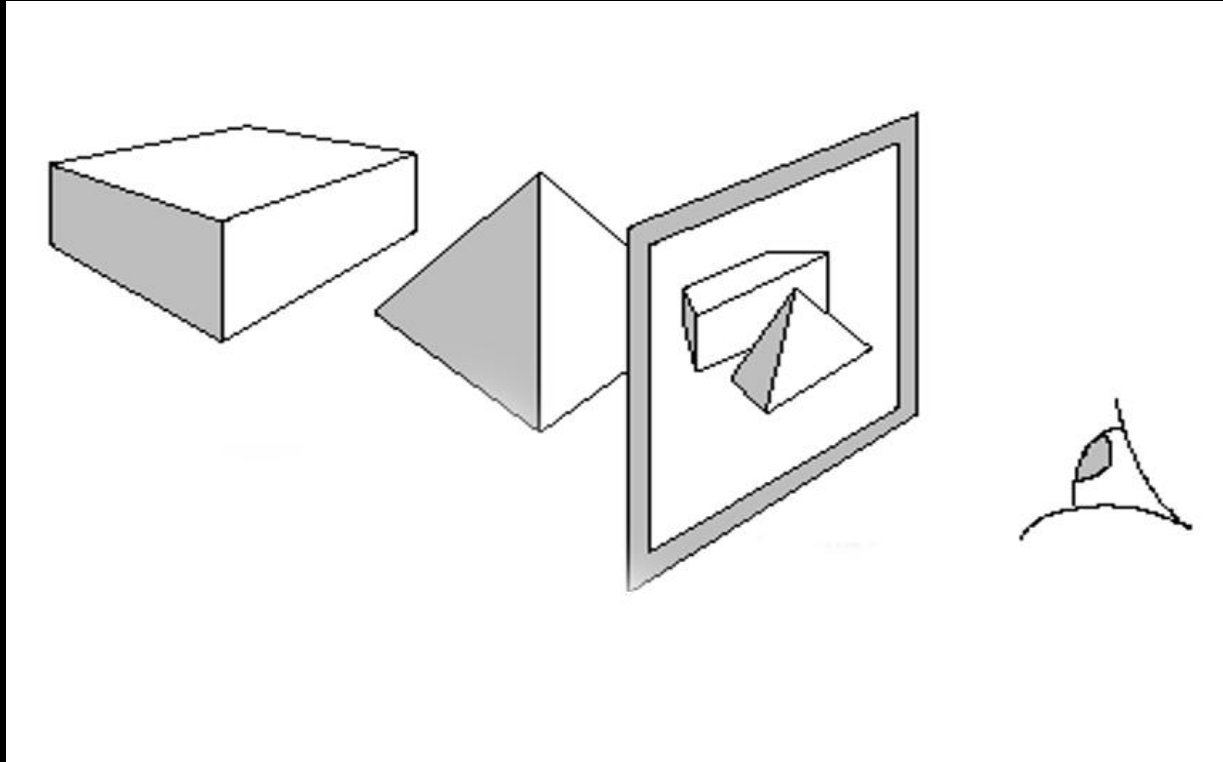
Unit 7

Section 1: Visible Surface Detection, Object Space method, Image space method

Visible Surface Detections



Visible Surface Detections



Visible Surface Detections

- Visible surface detection, also known as hidden surface removal, is a fundamental concept in computer graphics that involves determining which surfaces of a 3D scene are visible to the viewer and should therefore be rendered
- The goal is to avoid rendering surfaces that are occluded by other objects or surfaces, as rendering them would be wasteful and could lead to incorrect images.

Visible surface detection methods are broadly *classified* according to whether they deal with objects or with their projected images.

1. Object Space Method
2. Image Space method
3. List Priority Algorithms

Visible Surface Detections: Classification

✓ Object-Space Methods (OSM):

An object-space method compares objects and parts of objects to each other within the scene definition to determine which surfaces, as a whole, we should label as visible. E.g. Back-face detection method.

✓ Image-Space Methods (ISM):

In an image-space algorithm, visibility is decided point by point at each pixel position on the projection plane. Most visible-surface algorithms use image-space methods. E.g. Depth-buffer method, Scan-line method, Area-subdivision method.

✓ List Priority Algorithms

This is a hybrid model that combines both object and image precision operations. Here, depth comparison & object splitting are done with object precision and scan conversion (which relies on ability of graphics device to overwrite pixels of previously drawn objects) is done with image precision. E.g. Depth-Sorting method, BSP-tree method.



Visible Surface Detections : Object Space Method

Compiled by ab

- Algorithms to determine which parts of the shapes are to be rendered in 3D coordinates.
- Methods based on comparison of objects for their 3D positions and dimensions with respect to a viewing position.
- For N objects, may require $N*N$ comparison operations.
- Efficient for small number of objects but difficult to implement.
- Depth sorting, area subdivision methods.

Visible Surface Detections : Object Space Method



AMBITION GURU

- Deals with object definitions directly.
- Compare objects and parts of objects to each other within the scene definition to determine which surfaces, as a whole, we should label as visible.
- It is a continuous method.
- Compare each object with all other objects to determine the visibility of the object parts.



Visible Surface Detections : Object Space Method

Compiled by

Advantages of object-space methods:

- Object-space methods are generally more efficient than image space methods, as they eliminate hidden lines and surfaces before rendering the image.
- Object-space methods are also more accurate than image space methods, as they take into account the actual geometry of the scene.
- Object space methods can be used to perform other operations, such as collision detection and physics simulations.

Disadvantages of object space methods:

- Object space methods can be complex and difficult to implement, especially for scenes with complex geometry.
- Object space methods can be computationally expensive, as they require analyzing the relationships between the objects in the scene.

Visible Surface Detections : Image Space Method

Compiled by ab

- Deals with the projected images of the objects and not directly with objects.
- Visibility is determined point by point at each pixel position on the projection plane.
- It is a discrete method.
- Accuracy of the calculation is bounded by the display resolution.
- A change of display resolution requires re-calculation



Visible Surface Detections : Image Space Method

Compiled by

- Based on the pixels to be drawn on 2D. Try to determine which object should contribute to that pixel.
- Running time complexity is the number of pixels times number of objects.
- Space complexity is two times the number of pixels:
 - One array of pixels for the frame buffer
 - One array of pixels for the depth buffer
- Coherence properties of surfaces can be used.
- Depth-buffer and ray casting methods.

Visible Surface Detections : Image Space Method

Advantages of Image Space Methods:

- Image-space methods are generally easier to implement than object-space methods, as they only require analyzing the rendered image.
- Image-space methods are also more flexible than object-space methods, as they can be applied to any rendering technique.
- Image-space methods can be used to create artistic effects such as depth-of-field and motion blur.

Disadvantages of Image Space Methods:

- Image-space methods are less accurate than object-space methods, as they rely on approximations of the scene geometry.
- Image-space methods can be computationally expensive, especially for scenes with many objects or complex lighting.

Compiled by ab

Unit 7

Section 2: Back Face Detection Method, Z-buffer

Compiled by ab

Back Face Detection Method (Plane Equation Method)

Back - Face Detection Method

Compiled by ab

- There are surfaces *which are facing the viewer (front faces) and there are surfaces which are opposite to the viewer (back faces).*
- Back face detection method is a fast and simple **object-space method** for identifying the back faces of a polyhedron.

Inside-outside test in Back Face Detection Method

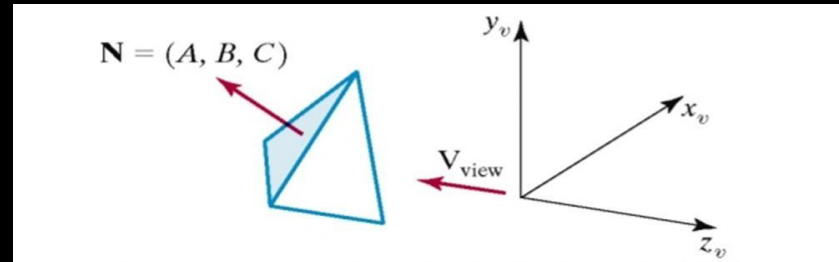
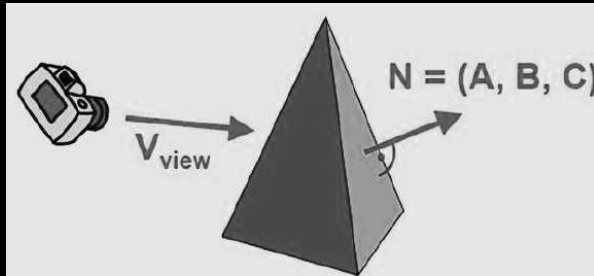
- Each surface has a normal vector. If this vector is pointing in the direction of the center of projection, it is a front face and can be seen by the viewer. If it is pointing away from the center of projection, it is a back face and cannot be seen by the viewer.
- The test is very simple, if the z component of the normal vector is positive, then, it is a back face. If the z component of the vector is negative, it is a front face.



Back -Face Detection Method

- Let N be normal vector to a polygon surface, which has Cartesian components (A, B, C) .
In general, if V is a vector in the viewing direction from the eye (or "camera") position, then this polygon is a back face.

if $V \cdot N > 0$.

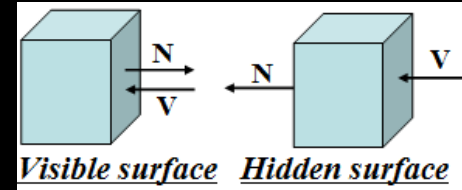


Back - Face Detection Method

A view vector V is constructed from any point on the surface to the viewpoint, the dot product of this vector and the normal N , indicates visible faces as follows:

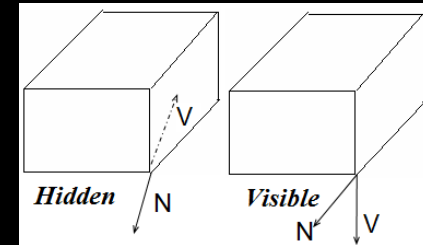
Case-I: (FRONT FACE)

If $V \cdot N < 0$ the face is visible *else* face is hidden

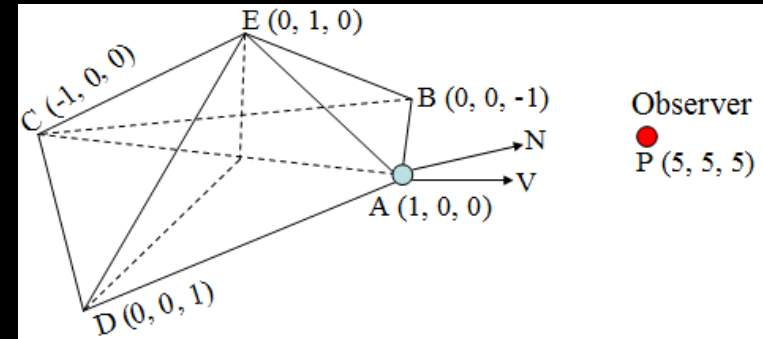


Case-II: (BACK FACE)

If $V \cdot N > 0$ the face is visible *else* face is hidden



Find the visibility for the surface AED in rectangular pyramid where an observer is at P (5, 5, 5).



Find the visibility for the surface AED where an observer is at P (5, 5, 5).

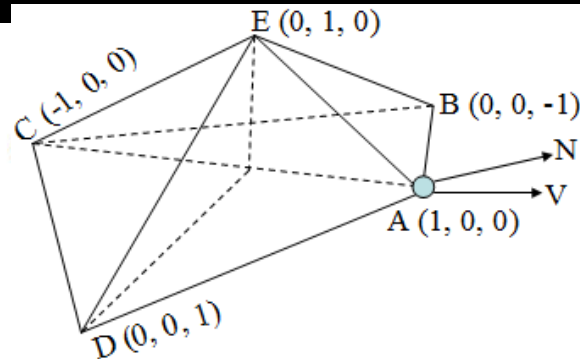
Here,

$$AE = (0-1) i + (1-0) j + (0-0) k = -i + j$$

$$AD = (0-1) i + (0-0) j + (1-0) k = -i + k$$

Step-1: Normal vector N for AED

$$\text{Thus, } N = AE \times AD = \begin{vmatrix} i & j & k \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{vmatrix} = i(1-0) - j(-1+0) + k(0+1) \\ = i + j + k$$



Observer
P (5, 5, 5)

Step-2: If observer at P(5, 5, 5) so we can construct the view vector V from view point A(1,0, 0) as:

$$V = PA = (1-5) i + (0-5) j + (0-5) k = -4i - 5j - 5k$$

Step-3: To find the visibility of the object, we use dot product of view vector V and normal vector N as:

$$V \cdot N = (-4i - 5j - 5k) \cdot (i + j + k) = -4-5-5 = -14 < 0$$

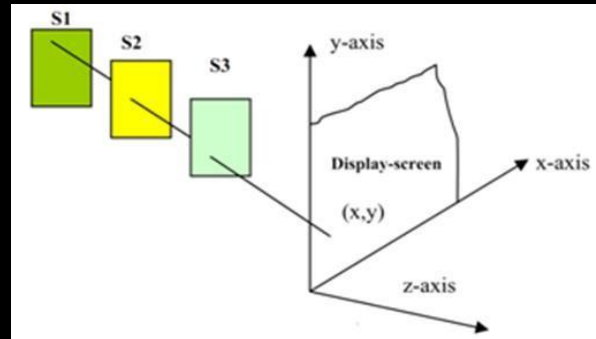
This shows that the surface is visible for the observer.

Compiled by ab

Depth Buffer (Z-Buffer) Method

Depth - Buffer (Z - Buffer Method)

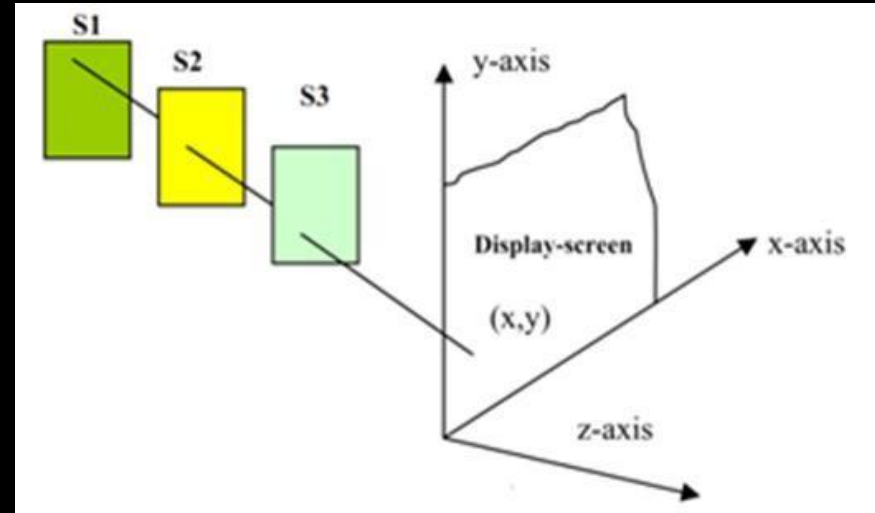
- A commonly used **image-space** approach to detecting visible surfaces is the **depth-buffer method**, which compares surface depths at each pixel position on the projection plane.
- Also called **z-buffer** method since depth usually measured along z-axis. This approach compares surface depths at each pixel position on the projection plane.
- Each surface of a scene is processed separately, one point at a time across the surface. And each (x, y, z) position on a polygon surface corresponds to the projection point (x, y) on the view plane.



Depth- Buffer (Z - Buffer Method)

This method requires two buffers:

- A **z-buffer** or **depth buffer**: Stores depth values for each pixel position (x, y) .
- **Frame buffer (Refresh buffer)**: Stores the surface-intensity values or color values for each pixel position.
- As surfaces are processed, the image buffer is used to store the color values of each pixel position and the z-buffer is used to store the depth values for each (x, y) position.



Depth - Buffer (Z - Buffer Method)

1. Initialize both, depth buffer and refresh buffer for all buffer positions (x, y),

$$\text{depth}(x, y) = 0 \quad \text{refresh}(x, y) = I_{\text{background}}$$

(where $I_{\text{background}}$ is the value for the background intensity.)

2. Process each polygon surface in a scene one at a time,

(Each surface listed in the polygon tables is then processed, one scan line at a time, calculating the depth (z-value) at each (x, y) pixel position.)

1. Calculate the depth z for each (x, y) position on the polygon.

(The calculated depth is compared to the value previously stored in the depth buffer at that position.)

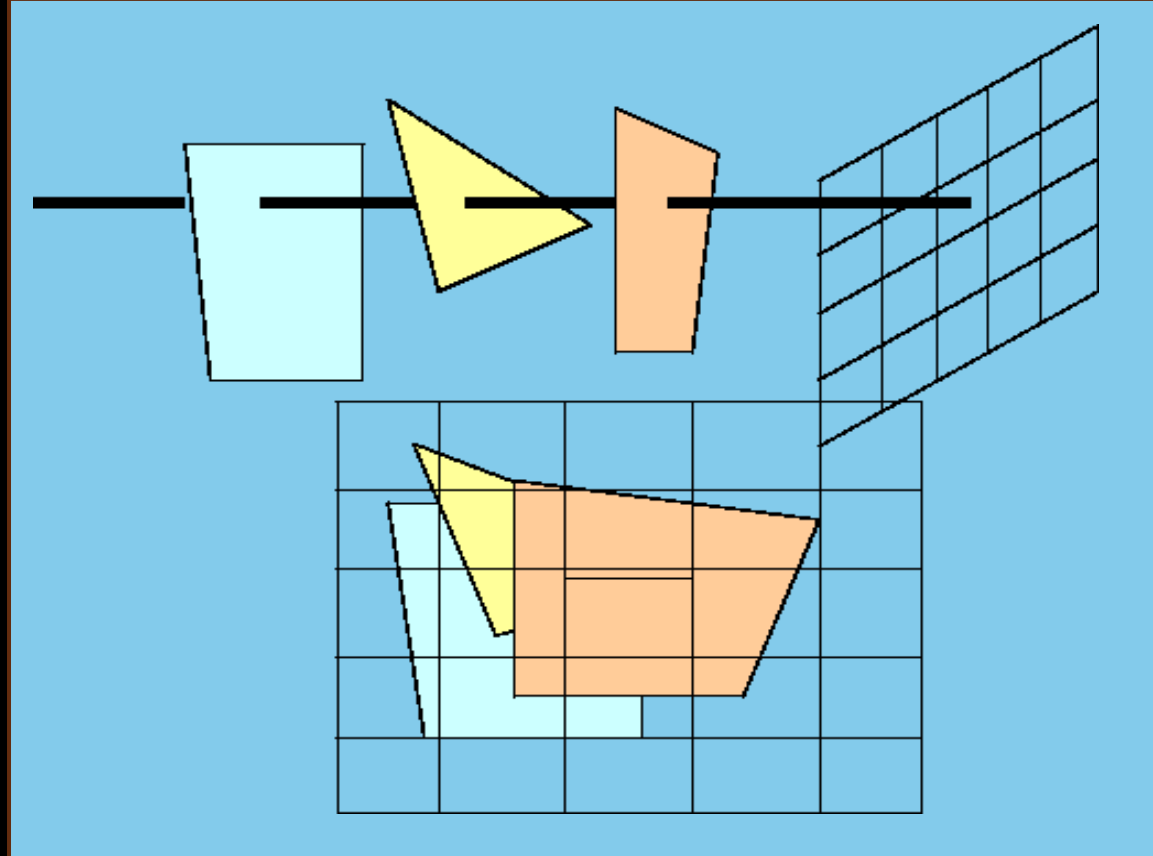
2. If $Z > \text{depth}(x, y)$, then set

$\text{depth}(x, y) = z$ (If the calculated depth is greater than the value stored in the depth buffer, the new depth value is stored,)

$\text{refresh}(x, y) = I_{\text{surf}}(x, y)$, (where $I_{\text{surf}}(x, y)$ is the intensity value for the surface at pixel position (x, y).

3. After all pixels and surfaces are compared, draw object using X,Y,Z from depth and intensity refresh buffer.

Depth - Buffer (Z - Buffer Method)



Depth - Buffer (Z - Buffer Method)

Advantages

- It is easy to implement.
- It reduces the speed problem if implemented in hardware.
- It processes one object at a time.

Disadvantages

- It requires large memory.
- It is time consuming process.

Compiled by ab

Unit 7

Section 3: A Buffer Method, Scan line method

Compiled by ab

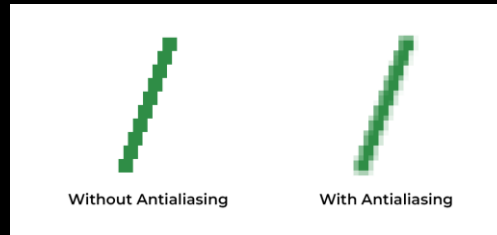
A- Buffer Method

A - Buffer Method

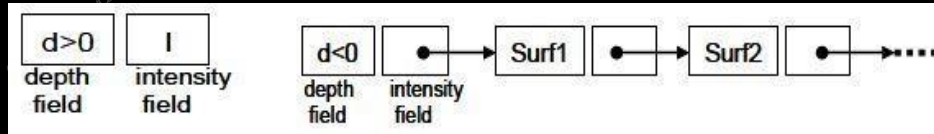
- The **A-buffer** (anti-aliased, area-averaged, accumulation buffer) is an extension of the ideas in the **depth-buffer** method (other end of the alphabet from "z-buffer").
- *A drawback of the **depth-buffer** method is that it deals only with opaque(Solid) surfaces and cannot accumulate intensity values for more than one transparent surfaces.*
- The **A-buffer** method is an extension of the depth-buffer method.
- The **A-buffer** is incorporated into the REYES ("Renders Everything You Ever Saw") 3-D rendering system.
- *The **A-buffer** method calculates the surface intensity for multiple surfaces at each pixel position, and object edges can be ant aliased.*

Each position in the A buffer has 2 fields :

1. Depth field
2. Surface data field or Intensity field



A - Buffer Method



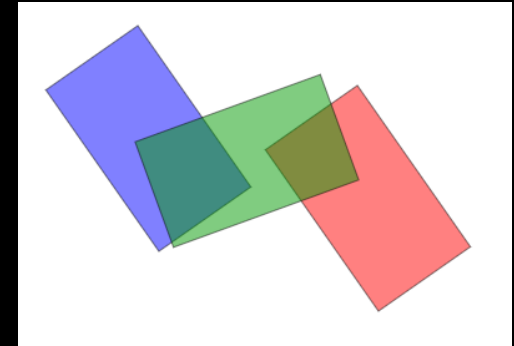
Each pixel position in the A-Buffer has two fields

Depth Field : stores a positive or negative real number

- Positive : **single** surface contributes to pixel intensity
- Negative : **multiple** surfaces contribute to pixel intensity

Intensity Field : stores surface-intensity information or a pointer value

- Surface intensity if single surface stores the RGB components of the surface color at that point and percent of pixel coverage
- Pointer value if multiple surfaces RGB intensity components
- Opacity parameter (percent of transparency)
- Percent of area coverage
- Surface identifier
- Other surface rendering parameters
- Pointer to next surface (Link List)

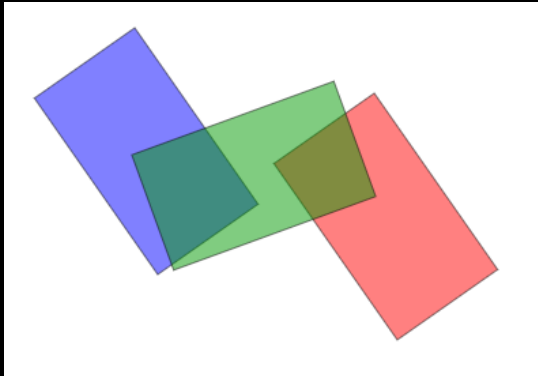


Multiple Surfaces Case

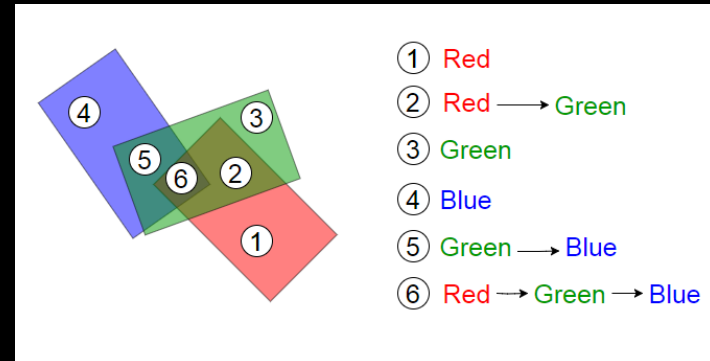
A - Buffer Method

- The **A-buffer** expands on the depth buffer method to allow transparencies. The key data structure in the A-buffer is the *accumulation buffer*.

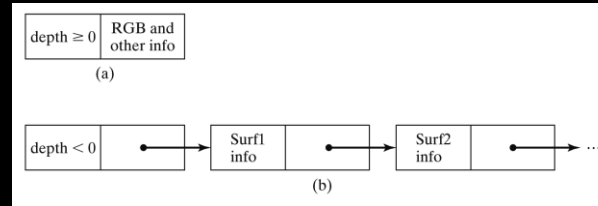
The usage of A-buffer algorithm for the transparent surfaces is as shown below :



On applying the A-buffer method on all the six surfaces indicated below, the corresponding colors are as :



A - Buffer Method



If depth is ≥ 0 , then the surface data field stores the depth of that pixel position as before (SINGLE SURFACE)

(If the depth field is positive, the number stored at that position is the depth of a single surface overlapping the corresponding pixel area. The intensity field then stores the RCB components of the surface color at that point and the percent of pixel coverage, as illustrated first figure.)

If depth < 0 then the data field stores a pointer to a linked list of surface data (MULTIPLE SURFACE)

(If the depth field is negative, this indicates multiple-surface contributions to the pixel intensity. The intensity field then stores a pointer to a linked list of surface data, as in second figure. Data for each surface in the linked list includes: RGB intensity components, opacity parameter (percent of transparency), depth, percent of area coverage, surface identifier, other surface-rendering parameters, and pointer to next surface)

- The A-buffer can be constructed using methods similar to those in the depth-buffer algorithm. Scan lines are processed to determine surface overlaps of pixels across the individual scan lines. Surfaces are subdivided into a polygon mesh and clipped against the pixel boundaries. Using the opacity factors and percent of surface overlaps, we can calculate the intensity of each pixel as an average of the contributions from the overlapping surfaces.

A - Buffer Method

Pros:

1. Accurate transparency rendering, including complex effects like refraction and translucency.
2. High-quality anti-aliasing for smoother edges and improved visual appeal.
3. Flexibility in handling various transparency effects, including overlapping objects.
4. No sorting required for transparent objects based on depth.

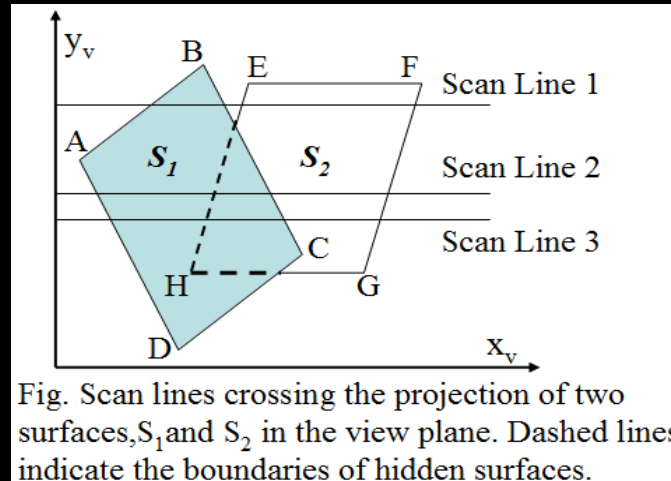
Cons:

1. Increased memory consumption due to storing additional pixel information.
2. Performance overhead during blending operations, impacting rendering speed.
3. Possibility of aliasing artifacts, especially in complex scenes.
4. Implementation complexity, requiring additional programming effort.

Scan Line Method

Scan-Line Method

- This **image-space** method for removing hidden surfaces is an extension of the **scan-line algorithm** for filling polygon interiors where, we deal with multiple surfaces rather than one.
- *Each scan line is processed with calculating the depth for nearest view for determining the visible surface of intersecting polygon. When the visible surface has been determined, the intensity value for that position is entered into the refresh buffer.*





Scan-Line Method

- To facilitate the search for surfaces crossing a given scan line, we can set up an **active list of edges** from information in the **edge table** that contain only edges that cross the current scan line, sorted in order of increasing x .
- In addition, we define a **flag** for each surface that is set **on** or **off** to indicate whether a position along a scan line is inside or outside of the surface. Scan lines are processed from left to right.
- At the leftmost boundary of a surface, the surface flag is turned on; and at the rightmost boundary, it is turned off.

Scan-Line Method

Compiled by ab

DATA STRUCTURE

- **A. Edge table** containing
 - Coordinate endpoints for each line in a scene
 - Inverse slope of each line
 - Pointers into polygon table to identify the surfaces bounded by each line
- **B. Surface table** containing
 - Coefficients of the plane equation for each surface
 - Intensity information for each surface
 - Pointers to edge table
- **C. Active Edge List**
 - To keep a trace of which edges are intersected by the given scan line

Note :

- The edges are sorted in order of increasing x
- Define flags for each surface to indicate whether a position is inside or outside the surface

Scan-Line Method: Algorithm

Compiled by ab

I. Initialize the necessary data structure

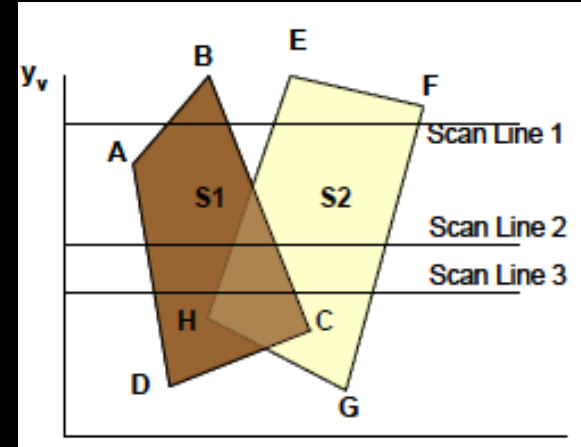
1. Edge table containing end point coordinates, inverse slope and polygon pointer.
2. Surface table containing plane coefficients and surface intensity
3. Active Edge List
4. Flag for each surface

II. For each scan line repeat

1. update active edge list
2. determine point of intersection and set surface on or off.
3. If flag is on, store its value in the refresh buffer
4. If more than one surface is on, do depth sorting and store the intensity of surface nearest to view plane in the refresh buffer

Scan-Line Method

- For scan line 1
 - The active edge list contains edges AB,BC,EH, FG
 - Between edges AB and BC, only *flags for s1 == on* and between edges EH and FG, only *flags for s2==on*
 - no depth calculation needed and corresponding surface intensities are entered in refresh buffer
- For scan line 2
 - The active edge list contains edges AD,EH,BC and FG
 - Between edges AD and EH, only the *flag for surface s1 == on*
 - Between edges EH and BC *flags for both surfaces == on*
 - Depth calculation (using plane coefficients) is needed.
 - In this example ,say s2 is nearer to the view plane than s1, so intensities for surface s2 are loaded into the refresh buffer until boundary BC is encountered
 - Between edges BC and FG flag for s1==off and *flag for s2 == on*
 - Intensities for s2 are loaded on refresh buffer
- For scan line 3
 - Same **coherent** property as scan line 2 as noticed from active list, so no depth

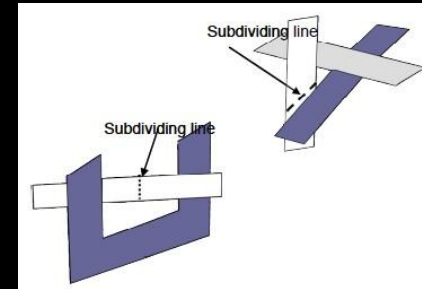
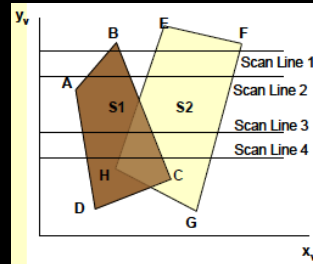
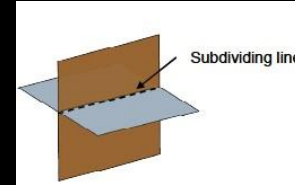


Scan-Line Method

Problem:

Dealing with **cut through** surfaces and **cyclic overlap** is problematic when used coherent properties

- **Solution:** Divide the surface to eliminate the overlap or cut through



Scan-Line Method

Pros:

- Efficiency: Efficient for rendering polygonal shapes, especially in scenes with simple geometry.
- Ease of Implementation: Relatively straightforward to implement compared to other rendering algorithms.
- Handling of Overlapping Polygons: Naturally handles overlapping polygons and renders visible portions based on depth information.
- Support for Shading and Texturing: Can incorporate shading models and texture mapping to enhance visual quality.

Cons:

- Limited to Planar Polygons: Primarily designed for rendering planar polygons, may struggle with non-planar or highly curved surfaces.
- Performance Degradation with Complex Scenes: May suffer performance degradation in scenes with many polygons or complex geometry.
- Edge Aliasing: Can produce edge aliasing artifacts, especially on diagonal or slanted edges.
- Limited Transparency Support: Handling transparent or translucent objects can be challenging and may require additional techniques.

Compiled by ab

Unit 7

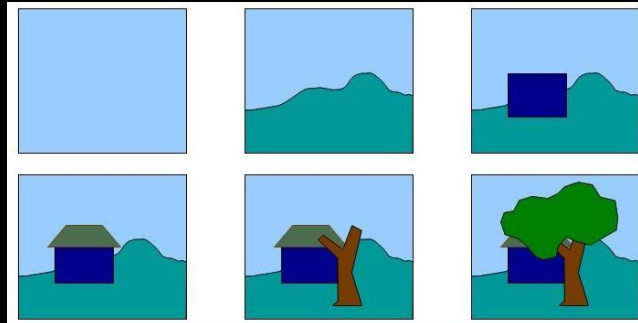
Section 4: Depth Sort (Painter Algorithm), BSP, Octree, Ray Tracing

Compiled by ab

Depth Sort (Painter Algorithm)

DEPTH SORT (Painter Algorithm)

- This method *uses both object space and image space* method.
- In this method the surface representation of 3D object are sorted in of decreasing depth from viewer.
- Then sorted surface are scan converted in order starting with surface of greatest depth for the viewer.

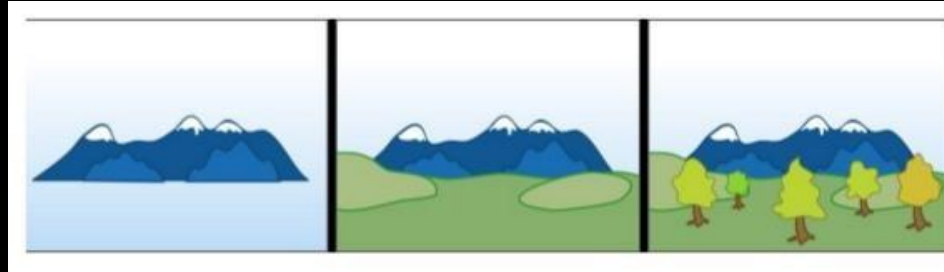


The conceptual steps that performed in depth-sort algorithm are

1. Sort all polygon surface according to the smallest (farthest) Z co-ordinate of each.
2. Resolve any ambiguity(doubt) this may cause when the polygons Z extents overlap, splitting polygons if necessary.
3. Scan convert each polygon in ascending order of smaller Z-coordinate i.e. farthest surface first (back to front).

DEPTH SORT (Painter Algorithm)

In this method, the newly displayed surface is partly or completely obscure the previously displayed surface. Essentially, we are sorting the surface into priority order such that surface with lower priority (lower z , far objects) can be obscured by those with higher priority (high z -value).

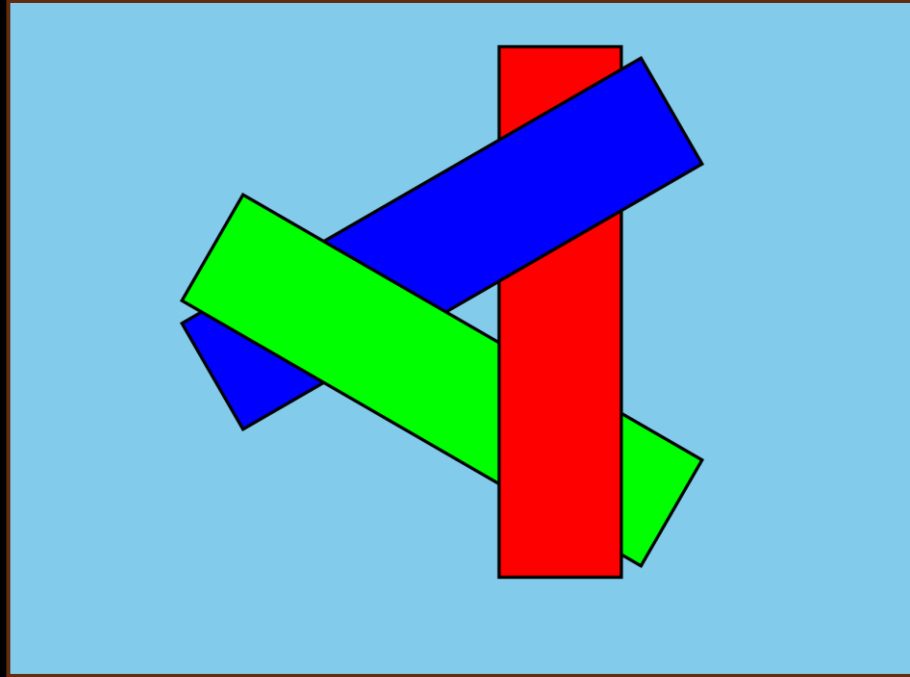


DEPTH SORT (Painter Algorithm)

Compiled by ab

- This algorithm is also called "**Painter's Algorithm**" as it simulates how a painter typically produces his painting by starting with the background and then progressively adding new (nearer) objects to the canvas.
- Thus, each layer of paint covers up the previous layer.
- Similarly, we first sort surfaces according to their distance from the view plane. The intensity values for the farthest surface are then entered into the refresh buffer. Taking each succeeding surface in turn (in decreasing depth order), we "**paint**" the surface intensities onto the frame buffer over the intensities of the previously processed surfaces.

DEPTH SORT (Painter Algorithm): Problem

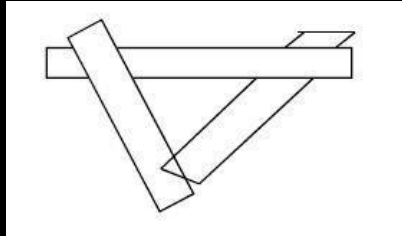


DEPTH SORT (Painter Algorithm)

Compiled by ab

Problem

- One of the major problem in this algorithm is intersecting polygon surfaces. As shown in fig. below.



- Different polygons may have same depth.
- The nearest polygon could also be farthest.

42

We cannot use simple depth-sorting to remove the hidden-surfaces in the images.

Solution: For intersecting polygons, we can split one polygon into two or more polygons which can then be painted from back to front. This needs more time to compute intersection between polygons. So it becomes complex algorithm for such surface existence.

Compiled by ab

Binary Space Partitioning (BSP)

Binary Space Partitioning (BSP)

- Binary space partitioning is a *3-D graphics programming technique of dividing a scene into two recursively using hyperplanes.*
- In other words, a 3-D scene is split in two using a 2-D plane, then that scene is divided in two using a 2-D plane, and so on. The resulting data structure is a binary tree, or a tree where every node has two branches.
- The technique is *widely used to speed up rendering of 3-D scenes, especially in games.*
- Binary space partitioning was developed in the context of 3D computer graphics, where the structure of a BSP tree allows spatial information about the objects in a scene that is useful in rendering, such as their ordering from front-to-back with respect to a viewer at a given location, to be accessed rapidly.

Binary Space Partitioning (BSP)

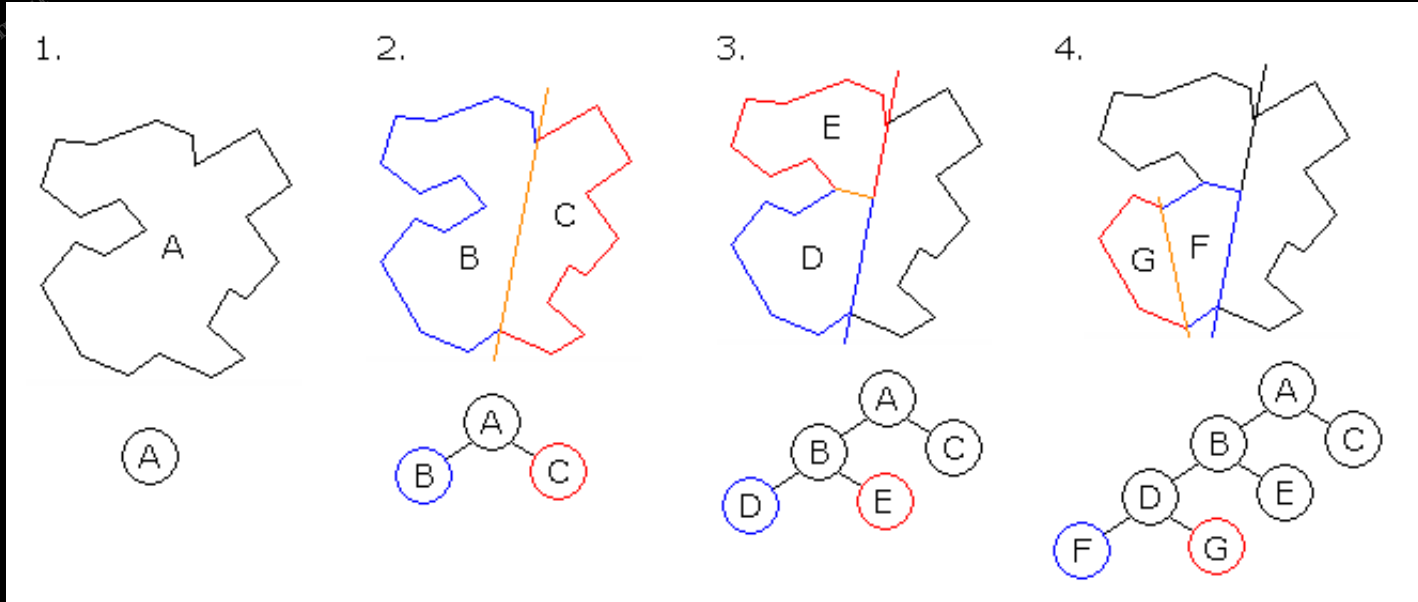


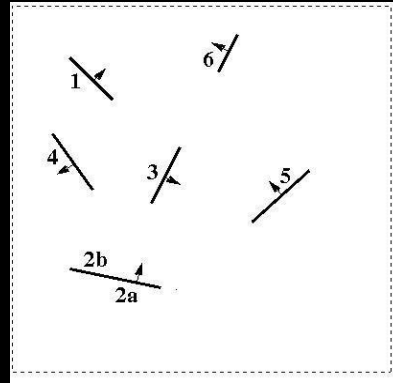
Fig. Binary Space Partitioning (BSP)

Building a BSP Tree (Recursive)

1. Start with a set of polygons and an empty tree
2. Select one of them and make it the root of the tree
3. Use its **plane to divide** the rest of the polygons in 3 sets: front, back, coplanar.
 1. Any polygon crossing the plane is split
4. Repeat the process recursively with the front and back sets, creating the front and back subtrees respectively

Building a BSP Tree (Recursive)

Compiled by ab



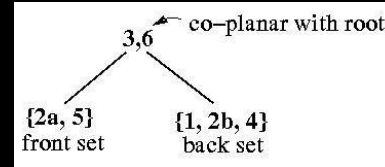
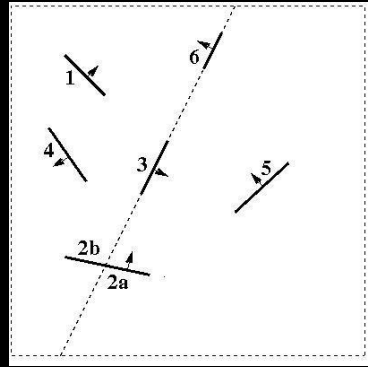
A set of polygons

The tree

{1, 2, 3, 4, 5, 6}

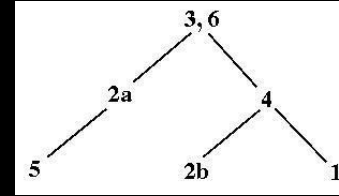
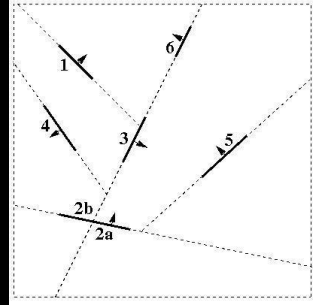
NOTE: If there are N set of polygons then there are N-1 planes

Building a BSP Tree (Recursive)



Select one polygon and partition the space and the polygons

Building a BSP Tree (Recursive)



Recursively partition each sub-tree until all polygons are used up

Applications of Binary Space Partitioning (BSP)

Compiled by a

1. **Rendering:** Efficient visibility determination in rendering algorithms.
2. **Collision Detection:** Quick detection of intersections between objects.
3. **Hidden Surface Removal:** Elimination of non-visible surfaces for rendering optimization.
4. **Spatial Partitioning:** Organization and indexing of spatial data in databases and GIS.
5. **Ray Tracing:** Acceleration of ray-object intersection tests for realistic image generation.
6. **Level Design in Games:** Partitioning game worlds for efficient rendering and occlusion culling.
7. **Constructive Solid Geometry (CSG):** Representation of complex geometries using Boolean operations.
8. **Robotics and Path Planning:** Partitioning of workspaces for collision-free path planning in robotics.

Pros of Binary Space Partitioning (BSP)

1. **Efficient Visibility Determination:** BSP trees facilitate fast determination of visible surfaces in rendering algorithms.
2. **Accurate Collision Detection:** BSP trees enable precise detection of intersections between objects, crucial for collision detection algorithms.
3. **Optimized Rendering:** BSP trees support hidden surface removal, improving rendering performance by eliminating non-visible geometry.
4. **Hierarchical Spatial Partitioning:** BSP trees provide a hierarchical structure for organizing spatial data efficiently.
5. **Adaptable to Dynamic Scenes:** BSP trees can adapt to changes in the scene dynamically, making them suitable for dynamic environments.
6. **Useful in Ray Tracing:** BSP trees accelerate ray-object intersection tests, enhancing the efficiency of ray tracing algorithms.
7. **Facilitates Level Design:** BSP trees aid in level design for games by partitioning game worlds into convex regions, optimizing rendering.

Cons of Binary Space Partitioning (BSP)

Compiled by ab

1. **Complexity:** Implementing and managing BSP trees can be complex, especially for dynamic scenes or irregular geometries.
2. **Memory Overhead:** BSP trees may consume significant memory, particularly for large or densely populated scenes.
3. **Construction Overhead:** Building BSP trees from a set of objects can involve high computational overhead, impacting performance.
4. **Difficulty in Dynamic Updates:** Modifying BSP trees dynamically can be challenging and may require costly reorganization.
5. **Potential for Imbalanced Trees:** BSP trees may become imbalanced, affecting query performance and memory usage.
6. **Limited in Non-Convex Scenes:** BSP trees are less effective in handling non-convex scenes, potentially leading to suboptimal visibility determination.
7. **Traversal Overhead:** Traversing BSP trees can incur overhead, especially in scenarios with deep trees or complex geometries.

Compiled by ab

Octree

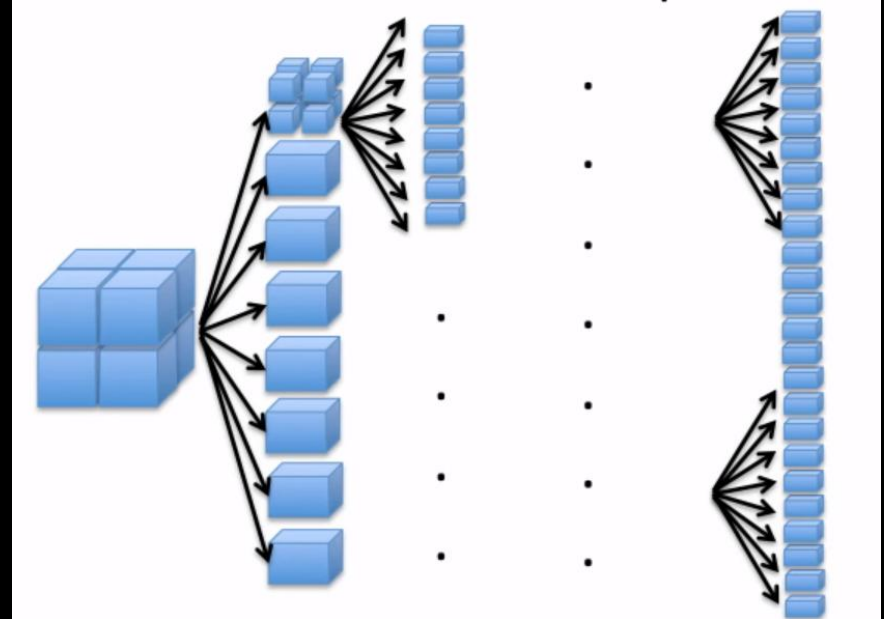
Octree

- **Octree** *is a tree data structure in which each internal node has exactly eight children.*
Octrees are most *often used to partition a three-dimensional space by recursively subdividing it into eight octants.*
- Octrees are the three-dimensional analog of quadtrees. The name is formed from *oct* + *tree*, but note that it is normally written "*octree*" with only one "t".
- Octrees are often used in 3D graphics and 3D game engines.

Octree

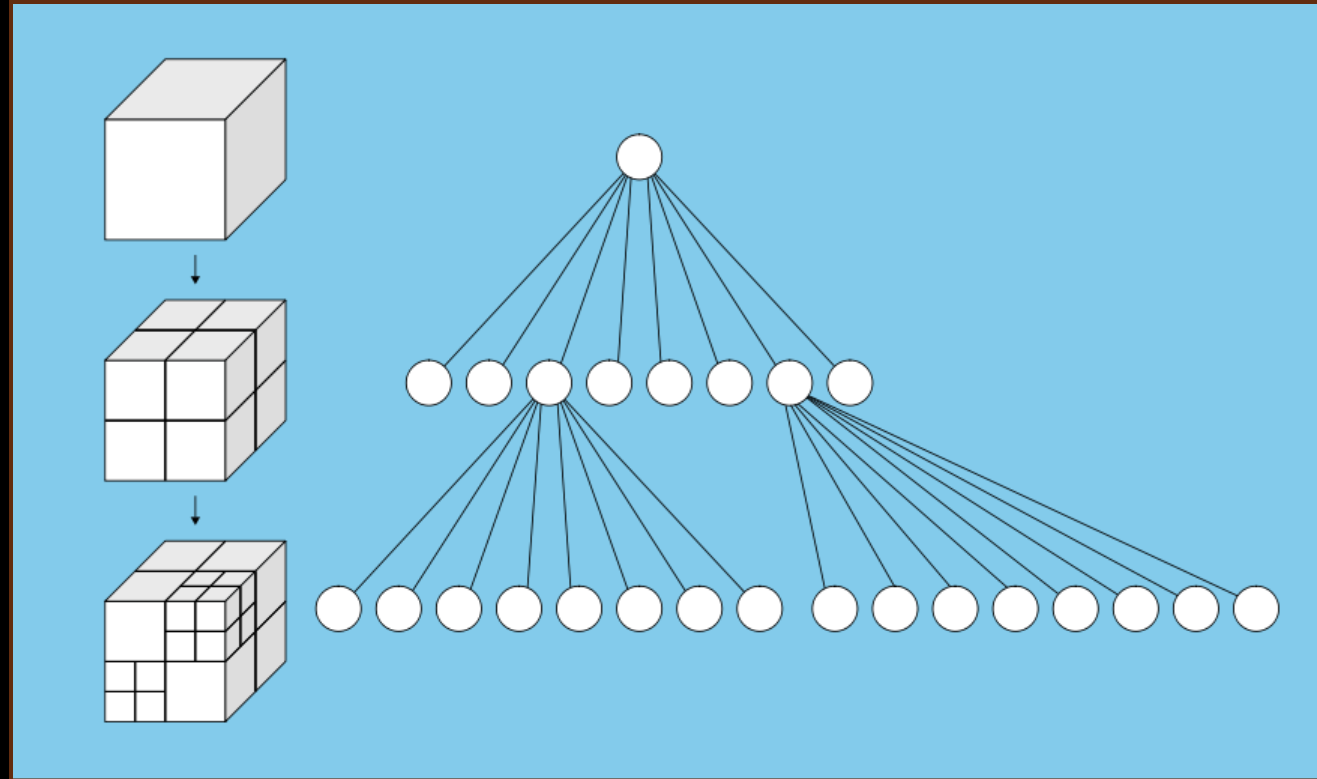
The Octree can be formed from 3D volume by doing the following steps:

1. Divide the current 3D volume into eight boxes.
2. If any box has more than one point, then divide it further into boxes.
3. Do not divide the box which has one or zero points in it.
4. Do this process repeatedly until all the box contains one or zero point in it.



Octree

Compiled by ab



Octree

Pros:

1. Efficient spatial partitioning.
2. Flexibility in adapting to dynamic scenes.
3. Memory efficiency, especially with sparse data.
4. Effective management of level of detail.
5. Hierarchical representation for fast spatial indexing.

Cons:

1. Memory overhead, especially for large scenes.
2. Complexity in implementation and management.
3. Performance trade-offs, especially in dynamic scenes.
4. Potential resolution issues in dense regions.
5. Construction overhead during tree building.

Compiled by ab

Ray Tracing

Ray tracing

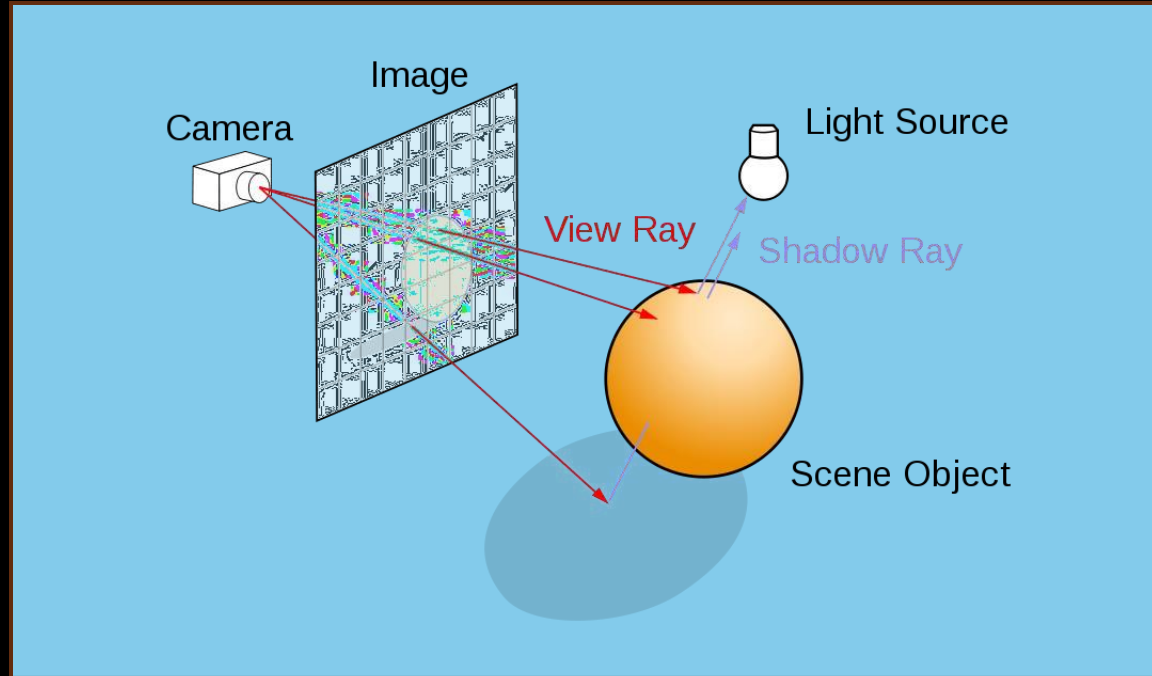
- Ray tracing is a rendering technique *for generating an image by tracing the path of light as pixels in an image plane and simulating the effects of its encounters with virtual objects.*
- The technique is capable of producing **a very high degree of visual realism**, usually higher than that of typical scanline rendering methods, but at a greater computational cost.
- This makes ray tracing best suited for applications where taking a relatively long time to render a frame can be tolerated, such as in still images and film and television visual effects, and more poorly suited for real-time applications such as video games where speed is critical.
- Ray tracing is capable of simulating a wide variety of *optical effects, such as reflection and refraction, scattering and dispersion phenomena (such as chromatic aberration).*

Ray tracing



AMBITION GURU

Compiled by ab



Ray tracing

Compiled by ab

- Optical ray tracing describes a method for producing visual images constructed in 3D computer graphics environments, with more photorealism than either ray casting or scanline rendering techniques.
- It works by tracing a path from an imaginary eye through each pixel in a virtual screen, and calculating the color of the object visible through it.

Ray tracing Applications:

1. Architecture: Architects can use ray tracing to visualize and present their designs with greater realism
2. Engineering: Engineers can use ray tracing to propose project concepts
3. Lighting design: Ray tracing can be used in theater and television lighting
4. Product development: Ray tracing can be used in CAD/CAM/CAE
5. Life sciences: Ray tracing can be used in medical and molecular applications
6. Energy: Ray tracing can be used in energy applications

Questions

1. Define visible surface detection. Explain in brief about different visible surface detection methods.
2. Define Back Face Detection Method. Find the visibility for the surface AED in rectangular pyramid where an observer is at P (5, 5, 5).
3. Define Z – Buffer Method. Write its algorithm along with its pros and cons.
4. Define A – Buffer Method. Write its algorithm along with its pros and cons.
5. Explain scan line method for visible surface detection.
6. Write short notes on:
 - i. Painter's Algorithm
 - ii. BSP
 - iii. Octree
 - iv. Ray Tracing

Compiled by ab

THANK YOU
Any Queries ?